

ANTIGRAVITY MODULAR IMPLEMENTATION PROMPTS

Smart Campus Lab & Resource Optimizer

15 standalone prompts for building the project module-by-module in Antigravity. Use the prompts one at a time, in order.

Smart Campus Lab & Resource Optimizer

This document contains **15 standalone prompts**. Paste them into Antigravity **one at a time, in order**. Each prompt tells Antigravity to first inspect the current project directory, preserve/extend existing work, and build only that module to completion (no placeholders, no TODOs) before stopping. The final prompt (Prompt 15) integrates and verifies everything.

Every prompt shares these **global conventions** — repeat them to Antigravity implicitly by referencing "the established conventions" in each prompt:

- Monorepo: `smart-campus-lab-optimizer/frontend`, `/backend`, `/docs`
- Backend: Node.js + Express + TypeScript, REST under `/api/v1`, MongoDB + Mongoose, JWT auth, RBAC, modular monolith (`routes` → `controller` → `service` → `repository` → `model` per domain)
- Frontend: React + Vite + TypeScript, Tailwind, shadcn/ui, React Router, TanStack Query, React Hook Form + Zod, Recharts, Lucide React
- Consistent API envelope: `{ success, message, data }` / `{ success, message, errors }` / paginated `{ success, data, pagination }`
- Service-abstraction pattern on both ends (Mock/Api on frontend; interface + local/basic implementation + future external adapter on backend)
- Roles: `student`, `faculty`, `admin`

PROMPT 1 — Project Foundation & Monorepo Setup

You are setting up the foundational structure for **Smart Campus Lab & Resource Optimizer**, a full-stack MERN application (React+TS/Vite frontend, Node+Express+TS backend, MongoDB/Mongoose database).

First, inspect the current project directory. If relevant files/folders already exist, preserve and build on them instead of overwriting. If nothing exists, initialize fresh.

Build in this module:

1. Root monorepo layout: `frontend/`, `backend/`, `docs/`, root `README.md` (stub, to be completed later), optional `docker-compose.yml` stub for MongoDB.

2. `backend/`: `package.json`, `tsconfig.json`, ESLint + Prettier config, folder skeleton exactly as:

src/config/ src/modules/ src/integrations/ src/middleware/ src/utils/

src/constants/ src/types/ src/seed/ src/app.ts src/server.ts

with barrel `index.ts` files where appropriate (no logic yet, but no dangling TODOs — an empty, valid, importable module is fine; a `// TODO: implement` file is not).

3. `frontend/`: Vite + React + TypeScript scaffold, Tailwind CSS installed and configured, shadcn/ui initialized, folder skeleton exactly as:

src/assets/ src/components/{common,layout,lab,booking,resource,monitoring,route,reports}/

src/layouts/{AuthLayout,StudentLayout,FacultyLayout,AdminLayout}/

src/pages/{auth,student,faculty,admin}/ src/routes/ src/services/

src/hooks/ src/types/ src/utils/ src/config/ src/data/mock/

4. Root-level `package.json` scripts (or a simple script using `concurrently`) to run frontend and backend together in dev.

5. `.gitignore` for both apps (node_modules, dist, .env, etc.).

6. `frontend/.env.example` and `backend/.env.example` with the variables:

- Frontend: `VITE\_API\_BASE\_URL`, `VITE\_DATA\_MODE`

- Backend: `PORT`, `NODE\_ENV`, `MONGODB\_URI`, `JWT\_SECRET`, `JWT\_EXPIRES\_IN`, `CLIENT\_URL`,
`SCHEDULING\_PROVIDER`, `ROUTING\_PROVIDER`, `CORE\_DATA\_PROVIDER`

Constraints: No business logic yet. No mock data yet. This module is purely structural. Verify both `npm install` and a basic `npm run dev` (frontend showing a blank Vite+Tailwind page, backend responding to a health-check route `GET /api/v1/health`) succeed before finishing.

PROMPT 2 — Backend Core Infrastructure & Database Models

Building on the existing project (inspect first, extend, do not overwrite Prompt 1's output), implement the backend's core infrastructure and full database schema.

Build in this module:

1. ``src/config/``: environment loader (validates required env vars fail fast), MongoDB connection module with retry/logging, app-wide constants (``roles.ts``: student/faculty/admin, ``bookingStatus.ts``: DRAFT/PENDING/QUEUED/CONFIRMED/ACTIVE/COMPLETED/CANCELLED/REJECTED).
2. ``src/utils/apiResponse.ts``: helper functions ``success(data, message)``, ``error(message, errors)``, ``paginated(data, pagination)`` implementing the standard envelope.
3. ``src/utils/asyncHandler.ts``: wraps async controllers to forward errors to Express error middleware.
4. ``src/middleware/errorHandler.ts``: centralized error handler mapping a custom ``AppError`` (with ``statusCode``, ``message``, ``errors[]``) to the standard error envelope; distinguishes validation (400) / auth (401) / forbidden (403) / not found (404) / server (500); never leaks stack traces outside development.
5. Full Mongoose models (one per domain module folder, e.g. ``src/modules/labs/lab.model.ts``) exactly matching this schema set:
 - **User**: name, email (unique, indexed), passwordHash, role (enum), department, profile{phone, avatarUrl}, timestamps
 - **Lab**: labId (unique), name, description, location, building, floor, capacity, operationalStatus (enum: available/occupied/maintenance/offline, indexed), maintenanceStatus, availableResources[ref Resource], timestamps
 - **Resource**: resourceId (unique), name, type, description, lab (ref Lab, indexed), operationalStatus, maintenanceStatus, availability(boolean), lastUpdated, timestamps
 - **Booking**: bookingId (unique), user (ref User, indexed), userRole, lab (ref Lab, indexed), date, startTime, endTime, purpose, requiredResources[ref Resource], allocatedResources[ref Resource], status (enum, indexed), queuePosition, requestsAhead, estimatedStart, schedulingMetadata (Mixed), cancellationReason, timestamps; compound index on (lab, date, status)
 - **MaintenanceRecord**: maintenancId (unique), targetType(enum lab/resource), target (refPath), affectedLab (ref Lab), description, status(enum: scheduled/in-progress/completed/cancelled), startDate, endDate, createdBy (ref User), timestamps
 - **Notification**: user (ref User, indexed), title, message, type, read(bool, default false), relatedEntityType, relatedEntityId, timestamps
 - **RouteResult**: booking (ref Booking), source, destination, nodes, edges, path, distanceMeters, estimatedTravelTimeMinutes, routeStatus, timestamps
 - **Alert**: severity, message, relatedEntityType, relatedEntityId, resolved, timestamps
6. ``src/app.ts``: Express app wired with helmet, cors (using ``CLIENT_URL``), JSON body parsing, request logging, the health-check route, and (placeholder mount points only — no routes yet) comments marking where module routers will attach. ``src/server.ts``: connects to Mongo, then starts the HTTP server.

Constraints: No routes/controllers/business logic beyond the health check yet — this module is data-layer and cross-cutting infrastructure only. Confirm the server boots and connects to MongoDB before finishing.

PROMPT 3 — Authentication & Role-Based Authorization

Building on the existing backend (inspect first), implement the full auth module and authorization middleware.

Build in this module:

1. ``src/modules/auth/``: controller, service, routes, Zod/express-validator validation schemas.
 - ``POST /api/v1/auth/register`` — creates a User (bcrypt-hashed password), returns JWT + safe user object.
 - ``POST /api/v1/auth/login`` — verifies credentials, returns JWT + user profile.
 - ``POST /api/v1/auth/logout`` — clears session/cookie as applicable.
 - ``GET /api/v1/auth/me`` — returns the authenticated user from the JWT.
2. ``src/modules/users/``: ``GET /api/v1/users/me``, ``PATCH /api/v1/users/me`` (update own profile fields).
3. ``src/middleware/authenticate.ts``: verifies JWT from Authorization header (or httpOnly cookie), attaches ``req.user``.
4. ``src/middleware/requireRole.ts``: ``requireRole('admin')``, ``requireRole('faculty','admin')``, etc. — returns 403 with the standard error envelope on mismatch.
5. Mount ``auth`` and ``users`` routers in ``app.ts`` under ``/api/v1``.
6. ``src/seed/seed.ts`` (initial version): creates exactly three demo accounts — one student, one faculty, one admin — with clearly documented (not hardcoded into UI) credentials, runnable via ``npm run seed``.
7. Backend tests (Jest/Vitest + Supertest): register/login success and failure cases, ``GET /auth/me`` requires a valid token, and a role-guarded dummy route proves ``requireRole`` blocks the wrong role.

Constraints: All password handling via bcrypt; JWT secret/expiry from env only; never return ``passwordHash`` in any response. Confirm login → protected route round-trip works via a manual curl/Postman-style check before finishing.

PROMPT 4 — Frontend Foundation & Design System

Building on the existing frontend scaffold (inspect first), implement the app shell, routing, layouts, and the full shared component library — no feature pages yet.

Build in this module:

1. Tailwind design tokens: restrained neutral palette + one accent color, spacing scale, typography scale; document them briefly in ``docs/design-system.md``.
2. ``src/components/common/``: Button, Input, Select, DatePicker, Modal, ConfirmDialog, Table, Pagination, Search, Filters, StatusBadge (with a fixed color mapping for every booking/lab/resource status), Skeleton, Spinner, ErrorMessage (with optional retry action), EmptyState (icon + message + CTA), Toast/notification system, Navbar, Sidebar (collapsible/mobile-drawer), Breadcrumbs. Every component must be genuinely reusable (typed props, no page-specific logic baked in).
3. ``src/layouts/``: ``AuthLayout``, ``StudentLayout``, ``FacultyLayout``, ``AdminLayout`` — each with its role-correct sidebar nav (per the nav lists below) and a top bar (user menu, notifications, logout).
 - Student/Faculty nav: Dashboard → Search Labs → My Bookings → Profile (Lab Details/Create Booking/Booking Details/Route are reached via navigation, not top-level nav items)
 - Admin nav: Dashboard → Labs → Resources → Maintenance → Bookings → Live Monitoring → Reports → Profile
 - Faculty/Student sidebars must never render admin-only items — build this from an explicit per-role nav config, not CSS hiding.
4. ``src/routes/``: ``AppRoutes`` (full route table for all pages listed in the master spec, using placeholder page components for now — each placeholder must render a simple "Page under construction" only as a *temporary* stand-in that later prompts will replace, and must be clearly marked so it's obvious nothing here is a final deliverable), ``ProtectedRoute`` (redirects unauthenticated users to ``/login``), ``RoleRoute`` (redirects wrong-role users to ``/unauthorized``).
5. ``src/services/api/``: Axios instance with base URL from ``VITE_API_BASE_URL``, request interceptor attaching the JWT, response interceptor handling 401 (logout + redirect) and normalizing the standard envelope.
6. ``src/services/`` factory pattern: for each future domain (auth, labs, bookings, resources, maintenance, monitoring, reports, routing) define the TypeScript **interface** now, plus an empty ``services/index.ts`` factory reading ``VITE_DATA_MODE`` — concrete Mock/Api implementations are filled in by later prompts.
7. ``/login``, ``/unauthorized`` pages, fully functional against the auth service interface (backed by a temporary minimal implementation if needed).
8. Global TanStack Query provider and Toast provider wired into the app root.

Constraints: This module must produce a navigable, role-aware shell with working login/logout and correctly gated routes, even though feature pages are placeholders. Confirm all four layouts render correctly and role gating works before finishing.

PROMPT 5 — Labs & Resources Domain Module (Backend + Shared Frontend Services)

Building on the existing project (inspect first), implement the Labs and Resources domains end-to-end — backend APIs, service abstractions, and the shared frontend components/services used by every role.

Build in this module:

Backend (`src/modules/labs/`, `src/modules/resources/`):

- `GET /api/v1/labs` (filters: date, time, capacity, equipment/resource type, status, free-text search), `GET /api/v1/labs/:labId`, `POST /api/v1/labs` (admin), `PATCH /api/v1/labs/:labId` (admin), `DELETE /api/v1/labs/:labId` (admin).
- `GET /api/v1/resources` (filters: lab, type, status, availability), `GET /api/v1/resources/:resourceId`, `POST /api/v1/resources` (admin), `PATCH /api/v1/resources/:resourceId` (admin — operational/maintenance state transitions), `DELETE /api/v1/resources/:resourceId` (admin).
- Full controller/service/repository split per module; request validation on all writes; `requireRole('admin')` on all write routes.

Frontend:

- `services/labs/`: `LabService` interface (`list`, `getById`, `create`, `update`), `MockLabService` (reading from `data/mock/labs.json`, simulated latency), `ApiLabService` (calls the backend above). Same pattern for `services/resources/` → `ResourceService`.
- `components/lab/`: LabCard, LabDetails, AvailabilityIndicator, LabFilterPanel — fully functional, typed, reusable.
- `components/resource/`: ResourceCard, ResourceTable, ResourceStatus.
- Realistic mock fixtures: at least 8 labs across varied capacity/location/status, at least 15 resources across varied type/condition/status.

Constraints: Do not build role-specific pages yet (Student Labs page, Admin Resources page, etc.) — only the domain services, backend endpoints, and shared components those pages will consume in later prompts. Confirm `MockLabService`/`MockResourceService` and `ApiLabService`/`ApiResourceService` both satisfy the same interface and return data in the standard envelope shape before finishing.

PROMPT 6 — Booking Module & Scheduling Abstraction

Building on the existing project (inspect first, and reuse the Labs/Resources domain from Prompt 5), implement the complete booking lifecycle and the pluggable scheduling/queue abstraction.

Build in this module:

Backend integrations layer (`src/integrations/scheduling/`):

- `SchedulingService` interface: `evaluate(request: BookingRequest): Promise<SchedulingResult>`, `cancel(bookingId): Promise<void>`.
- `SchedulingResult` contract: `{ bookingId, status, queuePosition, requestsAhead, estimatedStart, allocatedResources, schedulingMetadata }`.
- `BasicSchedulingService`: current implementation — checks lab/resource availability and time-slot overlap, assigns CONFIRMED/ACTIVE/QUEUED/PENDING status deterministically, computes a queue position and naive estimated start when queued.
- `ExternalSchedulingAdapter`: stubbed class implementing the same interface, not yet wired to a real external call, clearly commented as the future Team A integration point.
- `src/integrations/index.ts` factory selecting the implementation via `SCHEDULING_PROVIDER` env var; nothing else in the codebase imports a concrete scheduling class directly.

Backend booking module (`src/modules/bookings/`):

- `POST /api/v1/bookings` (creates booking, invokes `SchedulingService.evaluate`), `GET /api/v1/bookings` (own bookings, filterable by status), `GET /api/v1/bookings/:bookingId`, `PATCH /api/v1/bookings/:bookingId`, `PATCH /api/v1/bookings/:bookingId/cancel` (invokes `SchedulingService.cancel`), `GET /api/v1/admin/bookings` (admin-only, all users, filterable by lab/date/status).
- Full validation of the booking form fields server-side (date/time/purpose/resources), consistent with frontend validation in Prompt 8/9.

Frontend:

- `services/bookings/`: `BookingService` interface + `MockBookingService` + `ApiBookingService`, matching the `SchedulingResult` contract.
- `components/booking/`: `BookingCard`, `BookingForm` (React Hook Form + Zod), `BookingStatus`, `BookingStatusTracker`, `QueueStatus`, `BookingTimeline`, `BookingSummary` — all shared/reusable across Student and Faculty.
- Mock fixtures: bookings spanning every status (DRAFT/PENDING/QUEUED/CONFIRMED/ACTIVE/COMPLETED/CANCELLED/REJECTED) across multiple users and labs.

Constraints: The booking controller must never contain scheduling logic directly — it only calls the injected `SchedulingService`. Confirm a booking created through the API returns a fully populated `SchedulingResult` before finishing.

PROMPT 7 — Routing Integration Module

Building on the existing project (inspect first, reuse the Booking module from Prompt 6), implement the campus routing abstraction and route-visualization pieces.

Build in this module:

Backend (`src/integrations/routing/`, `src/modules/routing/`):

- `RoutingService` interface: `getRoute(bookingId): Promise<RouteResult>`.
- `RouteResult` contract: `{ source, destination, nodes[], edges[], path[], distanceMeters, estimatedTravelTimeMinutes, routeStatus }`.
- `MockRoutingService`: generates a believable campus topology (8–12 nodes representing buildings/entrances, with weighted edges), computes shortest path via a simple graph algorithm (Dijkstra or BFS on weighted edges) from a fixed "current location" node to the booked lab's node, and derives distance/time.
- `ExternalRoutingAdapter`: stubbed class implementing the same interface, clearly marked as the future Team B integration point.
- Register `RoutingService` in the `src/integrations/index.ts` factory via `ROUTING\_PROVIDER`.
- `GET /api/v1/routes/:bookingId` — delegates to the configured `RoutingService`.

Frontend:

- `services/routing/`: `RoutingService` interface + `MockRoutingService` + `ApiRoutingService`.
- `components/route/`: RouteTopology (custom SVG rendering of nodes/edges/highlighted path — chosen over a mapping library so the app needs zero external map API keys), RouteSummary, DistanceDisplay, ETADisplay.
- Mock fixtures: the same campus topology mirrored on the frontend for mock-mode parity with the backend's `MockRoutingService`.

Constraints: The Route feature must work fully offline/without any external map provider. Confirm `GET /api/v1/routes/:bookingId` returns a renderable path for a real seeded booking before finishing.

PROMPT 8 — Student Role Module

Building on the existing project (inspect first; reuse Labs/Resources/Bookings/Routing services and shared components from Prompts 5–7), implement every Student-facing page, replacing the placeholder routes from Prompt 4.

Build these pages under ``pages/student/`` and wire them into ``StudentLayout`` at these routes:

- ``/student/dashboard`` — Dashboard: available labs, upcoming bookings, current booking status, queue info, "Quick Book Lab" action, recent notifications, route shortcut.
- ``/student/labs`` — LabSearch: search + ``LabFilterPanel`` (date/time/capacity/equipment) over ``LabService``.
- ``/student/labs/:labId`` — LabDetails: full lab info, ``AvailabilityIndicator``, associated resources, "Book this lab" CTA.
- ``/student/bookings/create`` — CreateBooking: multi-step ``BookingForm`` (lab pre-filled if navigated from LabDetails → date → time slot → resources → purpose → review → confirm), full Zod validation, submits via ``BookingService``.
- ``/student/bookings`` — MyBookings: tabs/filters for upcoming/pending/queued/active/completed/cancelled, using ``BookingCard``.
- ``/student/bookings/:bookingId`` — BookingDetails: ``BookingSummary``, ``BookingStatusTracker``, ``QueueStatus``, allocated resources, lab info, ``RouteSummary`` with a link to the Route page.
- ``/student/route/:bookingId`` — Route: full ``RouteTopology`` + ``RouteSummary``.
- ``/student/profile`` — Profile: view/edit own profile via the users service.

Constraints: Every page must implement loading (skeletons matching layout), empty, error (with retry where retryable), validation, and permission states. No page may import mock JSON directly — only through the service interfaces from earlier prompts. Confirm the full student demo flow (login → dashboard → search/filter → lab details → create booking → booking details → route) works in mock mode before finishing.

PROMPT 9 — Faculty Role Module

Building on the existing project (inspect first; reuse everything from Prompts 5–8 — Faculty pages should be thin wrappers around the same shared components/services used by Student, with faculty-scoped routes/layout), implement every Faculty-facing page, replacing the placeholder routes from Prompt 4.

Build these pages under ``pages/faculty/``, mirroring Student's structure and wired into ``FacultyLayout``:

- ``/faculty/dashboard`` (adds emphasis on active bookings alongside the standard dashboard widgets)
- ``/faculty/labs``, ``/faculty/labs/:labId``
- ``/faculty/bookings/create``
- ``/faculty/bookings`` (upcoming/active/queued/completed/cancelled)
- ``/faculty/bookings/:bookingId``
- ``/faculty/route/:bookingId``
- ``/faculty/profile``

Explicit constraint: Do not duplicate ``BookingForm``, ``BookingCard``, ``LabCard``, etc. — import and reuse the exact same components built in Prompts 5–8, only the page-level composition and layout differ. Faculty must not be able to reach or see any Admin-only navigation item, page, or component. Confirm the faculty sidebar renders no admin items and that navigating directly to an ``/admin/*`` URL as a faculty user redirects to ``/unauthorized`` (enforced by both ``RoleRoute`` and the backend) before finishing.

PROMPT 10 — Admin Role Module (Labs, Resources, Maintenance, All Bookings)

Building on the existing project (inspect first; reuse Labs/Resources/Bookings services from Prompts 5–6), implement the Admin operational pages, replacing the placeholder routes from Prompt 4. (Monitoring and Reports are handled separately in Prompt 11.)

Build these pages under ``pages/admin/``, wired into ``AdminLayout``:

- ``/admin/dashboard`` — total labs, available/occupied/under-maintenance counts, active/pending bookings, queue size, resource availability summary, alerts, utilization summary.
- ``/admin/labs`` — table of all labs (id/name/capacity/location/status) with filters; row click → details.
- ``/admin/labs/:labId`` — Lab details + edit form (``PATCH /labs/:labId``) + list of associated resources.
- ``/admin/resources`` — ``ResourceTable`` with filters (type/lab/status/availability), "Add Resource" CTA.
- ``/admin/resources/create`` — `AddResource` form.
- ``/admin/resources/:resourceId`` — Resource details, current status, assigned lab, last updated, operational-state and maintenance-state action buttons with confirmation feedback (toast + inline status update).
- ``/admin/maintenance`` — Maintenance list (affected resource/lab, status) with status-transition actions (scheduled → in-progress → completed/cancelled), backed by ``src/modules/maintenance/`` backend (``GET/POST /maintenance``, ``GET/PATCH /maintenance/:maintenancId``).
- ``/admin/bookings`` — All bookings across all users, filterable by lab/date/status, row click → booking details (reusing the ``BookingDetails`` composition from the Student module where sensible).
- ``/admin/profile``

Constraints: Every write action (lab update, resource create/edit/state change, maintenance transition) must call a real backend endpoint enforcing ``requireRole('admin')``, and must show success/error feedback. Confirm an admin can create a resource, transition it into maintenance, and see that reflected on ``/admin/resources`` and ``/admin/maintenance`` before finishing.

PROMPT 11 — Monitoring & Reports Module

Building on the existing project (inspect first; reuse Labs/Resources/Bookings data from prior prompts), implement Admin Live Monitoring and Reports.

Build in this module:

Backend (`src/modules/monitoring/`, `src/modules/reports/`):

- `GET /api/v1/monitoring/overview`, `/monitoring/labs`, `/monitoring/resources`, `/monitoring/alerts` — all admin-only, aggregating current lab/resource/booking/alert state.
- `GET /api/v1/reports/dashboard`, `/reports/lab-utilization` (date range + lab filters), `/reports/resource-utilization`, `/reports/bookings` (counts by status over time) — all admin-only, computed via Mongoose aggregation pipelines against real seeded/created data.

Frontend:

- `services/monitoring/` and `services/reports/`: interface + Mock + Api implementations.
- `/admin/monitoring`: `components/monitoring/` — MonitoringCard, StatusOverview, AlertList; polls via TanStack Query `refetchInterval` (e.g. 20s) and shows a "last updated" timestamp; structure the service call so the polling mechanism is isolated and swappable for a future WebSocket/SSE push without touching the components.
- `/admin/reports`: `components/reports/` — ChartContainer, UtilizationChart, BookingChart (via Recharts), ReportFilterPanel (date range, lab, resource), DataTable for anything charts alone don't cover (e.g., raw counts table alongside each chart).

Constraints: Reports must be driven by real query results (not hardcoded numbers) so they change correctly as bookings/resources change. Confirm the monitoring page visibly updates on its poll interval and reports respond correctly to filter changes before finishing.

PROMPT 12 — Mock Data & Database Seeding Module

Building on the existing project (inspect first; you now have every domain from Prompts 2–11 available), finalize the realistic dataset on both sides so the whole application looks fully populated and demo-ready.

Build in this module:

1. ``frontend/src/data/mock/``: expand/finalize `labs.json`, `resources.json`, `bookings.json`, `maintenance.json`, `notifications.json`, and the routing topology fixture, ensuring every ``Mock*Service`` from earlier prompts consumes this data consistently and that the data supports meaningful dashboards/reports/status-badge variety out of the box.
2. ``backend/src/seed/seed.ts``: expand/finalize to create — multiple students, multiple faculty, one admin; multiple labs across varied capacity/location/operational-status; multiple resources across varied type/condition/status; bookings across every status (upcoming, queued, active, completed, cancelled, rejected) spread across several labs/users/dates; several maintenance records; several notifications per user. Script must be idempotent or clearly documented as destructive-and-reseeding (``npm run seed`` drops/recreates demo collections safely in a dev DB).
3. Verify: after ``npm run seed``, the Admin Dashboard, Monitoring, and Reports pages (in API mode) render non-trivial charts/tables, and Student/Faculty accounts each have a realistic mix of booking statuses to browse.

Constraints: Frontend mock data and backend seed data should tell "the same story" (comparable variety and volume) even though they are separate datasets. Confirm both datasets independently make every dashboard/report page look realistic before finishing.

PROMPT 13 — Testing & Cross-Cutting Error Handling

Building on the existing project (inspect first — everything should now be feature-complete from Prompts 1–12), add the testing suite and close any remaining gaps in error/loading/empty/validation states across the app.

Build in this module:

1. Backend tests (Jest/Vitest + Supertest): auth flows, booking creation (asserting `SchedulingService`` is invoked and a full `SchedulingResult`` is returned), role-authorization middleware (a student hitting an admin-only route gets 403), and at least one test per domain module's primary read/write endpoint.
2. Frontend unit tests (Vitest + React Testing Library): `BookingForm`` validation behavior, `StatusBadge`` status→color mapping, `LabFilterPanel`` filter application, and at least one hook test (e.g. `useBookings``).
3. One Playwright end-to-end test: login as student → search/filter labs → open lab details → create a booking → land on booking details showing status, queue position, allocated resources, estimated start, and a route summary link.
4. Sweep every page built in Prompts 8–11 and confirm each has: a matching-layout loading skeleton, an `EmptyState`` for the zero-results case, an `ErrorMessage`` with retry for retryable failures, field-level validation errors surfaced from both client (Zod) and server, and correct behavior when hit with an unauthorized role (redirect) or unauthorized action (backend 403 surfaced as an inline message).
5. Confirm the global 401 interceptor correctly logs out and redirects with a "session expired" message when a token expires mid-session.

Constraints: This module is verification-heavy — do not introduce new features, only tests and gap-fixes to existing pages/endpoints. Confirm all test suites pass (`npm run test`` in both frontend and backend) before finishing.

PROMPT 14 — Documentation Module

Building on the existing, now feature-complete project (inspect first), produce final documentation.

Build in this module:

1. Root `README.md`: project description, architecture summary (with the mock/api and integration-adapter pattern explained), tech stack, folder structure overview, setup instructions for frontend and backend, environment variable reference (linking to the `.env.example` files), how to seed the database, demo credentials for student/faculty/admin, how to switch `VITE_DATA_MODE` between `mock` and `api`, a full API endpoint reference table, and a dedicated section listing the three future integration points (Team A scheduling, Team B routing, Team C core data) with the exact file path to edit for each.
2. `docs/design-system.md`: finalize with the actual tokens/components used.
3. `docs/architecture.md`: diagram (ASCII or Mermaid) of frontend → service layer → backend → integrations → database, matching the actual implementation.
4. Inline code comments on every integration-adapter stub (`ExternalSchedulingAdapter`, `ExternalRoutingAdapter`, `ExternalCoreDataAdapter` if present) explaining exactly what a future implementer needs to do to wire in the real external service.

Constraints: Documentation must describe what was actually built, not the aspirational spec — verify every command and env var mentioned in the README actually works before finishing.

PROMPT 15 — FINAL INTEGRATION PROMPT (run last)

You now have a fully built, module-by-module implementation of **Smart Campus Lab & Resource Optimizer** across Prompts 1–14: monorepo foundation, backend core + database, authentication/authorization, frontend foundation + design system, Labs/Resources domain, Booking + Scheduling abstraction, Routing abstraction, Student module, Faculty module, Admin module, Monitoring + Reports, mock data + seeding, testing, and documentation.

Your task now is integration and full end-to-end verification — not new features.

1. **Inspect the entire project** top to bottom (both ``frontend/`` and ``backend/``) and reconcile anything inconsistent between modules built in separate passes — e.g., mismatched types between a service interface and its Mock/Api implementations, route paths that drifted from the spec, or a component built in one prompt that wasn't actually wired into the page built in a later prompt. Fix these silently as part of integration; do not leave known inconsistencies.

2. **Verify mock mode end-to-end:** with ``VITE_DATA_MODE=mock`` and no backend running, walk the complete demonstration flow purely in the frontend:

Login → role-based Dashboard → Search Lab → Filter Lab → Open Lab Details → Create Booking → Submit Booking → View Booking Details (status, queue position, allocated resources, estimated start, route summary) → Open Route Page → (switch account) Login as Admin → see the same operational data via Monitoring → View Reports.

3. **Verify API mode end-to-end:** run ``npm run seed`` against a local MongoDB, start the backend, set ``VITE_DATA_MODE=api``, and repeat the exact same demonstration flow against the real API, confirming parity with mock mode (same shape of data, same UI behavior).

4. **Verify role security is enforced twice:** for every admin-only route/action, confirm (a) the frontend hides/redirects correctly for non-admins, and (b) directly calling the backend endpoint as a student/faculty JWT independently returns 403 — the frontend gate must never be the only protection.

5. **Run the full test suite** (backend unit/integration, frontend unit, Playwright E2E) and fix any failures surfaced by integration.

6. **Run linting** on both apps and fix violations.

7. **Confirm the app boots cleanly** via the root dev script with zero required external services (mock scheduling/routing/core-data providers as defaults), and confirm switching every provider env var to its "external" stub does not crash the app (it should either no-op gracefully or clearly indicate the adapter isn't yet wired, never throw an unhandled exception).

Finally, produce a concise summary containing:

- Files created, grouped by frontend / backend / docs
- Major architecture decisions and why they were made
- Full list of implemented API endpoints
- Demo credentials (student / faculty / admin)
- How to run the frontend
- How to run the backend
- How to seed the database
- How to toggle mock vs. real API data mode
- The exact file to edit for each of the three future integration points (Team A scheduling, Team B routing, Team C core data)

This summary is the final deliverable of the entire build.